Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Ростовский государственный экономический университет (РИНХ)»

ОТЧЁТ

по лабораторной работе № 1.5

«Взаимодействие с пользователем»

по дисциплине «Разработка и сопровождение программных систем»

Вариант 13

Выполнил: студент группы ИСТ-321
Хасаев Умахан

Ростов-на-Дону
2026

Репозиторий с исходным кодом: https://github.com/warhogz/software-development-and-maintenance

Цель работы

Научиться обрабатывать действия пользователя в JavaFX-приложении: реагировать на выбор записи в таблице, добавлять, изменять и удалять записи с помощью диалоговых окон, проверять корректность пользовательского ввода.

Задание

1. Реакция на выбор районов в таблице.

2. Добавление функциональности кнопкам «Новый», «Изменить» и «Удалить».

3. Создание диалогового окна для изменения информации о районе.

4. Проверка пользовательского ввода.

Индивидуальное задание (вариант 13): предметная область «Области – Районы».

Таблица 1 «Области»: название, площадь, административный центр, глава.

Таблица 2 «Районы»: название, площадь, административный центр, глава.

Ход работы

Работа выполнена на основе проекта лабораторной работы 1.4. В проект добавлены FXML-файл диалогового окна DistrictEditDialog.fxml и его контроллер DistrictEditDialogController.

1. Реакция на выбор в таблице. В методе initialize() контроллера DistrictOverviewController на свойство selectedItemProperty() модели выбора таблицы установлен слушатель. При выборе строки вызывается метод showDistrictDetails(), который заполняет метки в правой части окна сведениями о районе, а при отсутствии выбора очищает их.

2. Функциональность кнопок. Кнопкам в файле DistrictOverview.fxml назначены обработчики через атрибут onAction (листинг 1). Метод handleDeleteDistrict() удаляет выбранный район из таблицы; если район не выбран, выводится предупреждение Alert. Методы handleNewDistrict() и handleEditDistrict() открывают диалоговое окно для нового или выбранного района (листинг 2).

Листинг 1 – Назначение обработчиков кнопкам в DistrictOverview.fxml

```
<ButtonBar AnchorPane.bottomAnchor="10.0" AnchorPane.rightAnchor="10.0">
  <buttons>
    <Button mnemonicParsing="false" onAction="#handleNewDistrict" text="Новый..." />
    <Button mnemonicParsing="false" onAction="#handleEditDistrict" text="Изменить..."
/>
    <Button mnemonicParsing="false" onAction="#handleDeleteDistrict" text="Удалить" />
  </buttons>
</ButtonBar>
```

Листинг 2 – Класс DistrictOverviewController

```
package ru.lab.regionapp.view;

import javafx.beans.binding.Bindings;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;
import javafx.scene.control.Label;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import ru.lab.regionapp.MainApp;
import ru.lab.regionapp.model.District;
import ru.lab.regionapp.model.Region;

/**
 * Контроллер для DistrictOverview.fxml
 */
public class DistrictOverviewController {

    @FXML
    private TableView<District> districtTable;
    @FXML
    private TableColumn<District, String> nameColumn;
    @FXML
    private TableColumn<District, String> regionColumn;

    @FXML
    private Label nameLabel;
    @FXML
    private Label areaLabel;
    @FXML
    private Label adminCenterLabel;
    @FXML
    private Label headLabel;
    @FXML
    private Label regionLabel;

    // Ссылка на главное приложение.
    private MainApp mainApp;
```

```
    /**
     * Конструктор вызывается раньше метода initialize().
     */
    public DistrictOverviewController() {
    }

    /**
     * Инициализация класса-контроллера. Этот метод вызывается автоматически
     * после того, как fxml-файл будет загружен.
     */
    @FXML
    private void initialize() {
        // Инициализация таблицы районов с двумя столбцами.
        nameColumn.setCellValueFactory(cellData ->
cellData.getValue().nameProperty());
        // Столбец «Область» обновляется, если у района сменили область.
        regionColumn.setCellValueFactory(cellData -> Bindings.createStringBinding(() -
> {
            Region region = cellData.getValue().getRegion();
            return region == null ? "" : region.getName();
        }, cellData.getValue().regionProperty()));

        // Очистка подробной информации о районе.
        showDistrictDetails(null);

        // Слушаем изменения выбора и при изменении отображаем подробную информацию о
районе.
        districtTable.getSelectionModel().selectedItemProperty().addListener(
                (observable, oldValue, newValue) -> showDistrictDetails(newValue));
    }

    /**
     * Вызывается главным приложением, которое даёт на себя ссылку.
     */
    public void setMainApp(MainApp mainApp) {
        this.mainApp = mainApp;

        // Добавление в таблицу данных из наблюдаемого списка.
        districtTable.setItems(mainApp.getDistrictData());
    }

    /**
     * Заполняет текстовые поля, отображая подробности о районе.
     * Если указанный район = null, то все текстовые поля очищаются.
     */
    private void showDistrictDetails(District district) {
        if (district != null) {
            nameLabel.setText(district.getName());
            areaLabel.setText(String.format("%.0f км²", district.getArea()));
            adminCenterLabel.setText(district.getAdminCenter());
            headLabel.setText(district.getHead());
            regionLabel.setText(district.getRegion() == null ? "" :
district.getRegion().getName());
        } else {
            nameLabel.setText("");
            areaLabel.setText("");
            adminCenterLabel.setText("");
            headLabel.setText("");
            regionLabel.setText("");
        }
    }

    /**
     * Вызывается, когда пользователь кликает по кнопке «Удалить».
     */
```

```
    @FXML
    private void handleDeleteDistrict() {
        int selectedIndex = districtTable.getSelectionModel().getSelectedIndex();
        if (selectedIndex >= 0) {
            districtTable.getItems().remove(selectedIndex);
        } else {
            showNoSelectionAlert();
        }
    }

    /**
     * Вызывается, когда пользователь кликает по кнопке «Новый...».
     * Открывает диалоговое окно с пустыми полями для нового района.
     */
    @FXML
    private void handleNewDistrict() {
        District tempDistrict = new District();
        boolean okClicked = mainApp.showDistrictEditDialog(tempDistrict);
        if (okClicked) {
            mainApp.getDistrictData().add(tempDistrict);
            districtTable.getSelectionModel().select(tempDistrict);
        }
    }

    /**
     * Вызывается, когда пользователь кликает по кнопке «Изменить...».
     * Открывает диалоговое окно для изменения выбранного района.
     */
    @FXML
    private void handleEditDistrict() {
        District selectedDistrict =
districtTable.getSelectionModel().getSelectedItem();
        if (selectedDistrict != null) {
            boolean okClicked = mainApp.showDistrictEditDialog(selectedDistrict);
            if (okClicked) {
                showDistrictDetails(selectedDistrict);
            }
        } else {
            showNoSelectionAlert();
        }
    }

    /**
     * Предупреждение, если в таблице ничего не выбрано.
     */
    private void showNoSelectionAlert() {
        Alert alert = new Alert(AlertType.WARNING);
        alert.initOwner(mainApp.getPrimaryStage());
        alert.setTitle("Нет выбора");
        alert.setHeaderText("Район не выбран");
        alert.setContentText("Пожалуйста, выберите район в таблице.");
        alert.showAndWait();
    }
}
```

3. Диалоговое окно. Метод showDistrictEditDialog() класса MainApp загружает DistrictEditDialog.fxml, создаёт модальное окно Stage (Modality.WINDOW_MODAL), передаёт контроллеру редактируемый район и список областей для выпадающего списка ComboBox, ожидает закрытия окна с помощью showAndWait() и возвращает признак нажатия кнопки OK (листинг 3).

Листинг 3 – Метод showDistrictEditDialog() класса MainApp

```
    /**
     * Открывает диалоговое окно для изменения информации о районе.
     * Если пользователь кликнул OK, изменения сохраняются в предоставленном
     * объекте района и возвращается значение true.
     *
     * @param district объект района, который надо изменить
     * @return true, если пользователь кликнул OK, в противном случае false.
     */
    public boolean showDistrictEditDialog(District district) {
        try {
            // Загружаем fxml-файл и создаём новую сцену для всплывающего диалогового
окна.
            FXMLLoader loader = new FXMLLoader();

loader.setLocation(MainApp.class.getResource("view/DistrictEditDialog.fxml"));
            AnchorPane page = (AnchorPane) loader.load();


            // Создаём диалоговое окно Stage.
            Stage dialogStage = new Stage();
            dialogStage.setTitle(district.getName() == null ? "Новый район" :
"Редактирование района");
            dialogStage.initModality(Modality.WINDOW_MODAL);
            dialogStage.initOwner(primaryStage);
            Scene scene = new Scene(page);
            dialogStage.setScene(scene);


            // Передаём район и список областей в контроллер.
            DistrictEditDialogController controller = loader.getController();
            controller.setDialogStage(dialogStage);
            controller.setRegions(regionData);
            controller.setDistrict(district);


            // Отображаем диалоговое окно и ждём, пока пользователь его не закроет.
            dialogStage.showAndWait();


            return controller.isOkClicked();
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }
```

4. Проверка пользовательского ввода. В контроллере диалогового окна метод isInputValid() проверяет, что заполнены все поля и выбрана область, площадь является положительным числом и меньше площади выбранной области. Если найдены ошибки, их список выводится в окне Alert, а данные не сохраняются (листинг 4).

Листинг 4 – Класс DistrictEditDialogController

```
package ru.lab.regionapp.view;


import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.Alert;
import javafx.scene.control.Alert.AlertType;
import javafx.scene.control.ComboBox;
```

```
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import ru.lab.regionapp.model.District;
import ru.lab.regionapp.model.Region;

/**
 * Окно для изменения информации о районе.
 */
public class DistrictEditDialogController {

    @FXML
    private TextField nameField;
    @FXML
    private TextField areaField;
    @FXML
    private TextField adminCenterField;
    @FXML
    private TextField headField;
    @FXML
    private ComboBox<Region> regionBox;

    private Stage dialogStage;
    private District district;
    private boolean okClicked = false;

    /**
     * Инициализирует класс-контроллер. Этот метод вызывается автоматически
     * после того, как fxml-файл будет загружен.
     */
    @FXML
    private void initialize() {
    }

    /**
     * Устанавливает сцену для этого окна.
     */
    public void setDialogStage(Stage dialogStage) {
        this.dialogStage = dialogStage;
    }

    /**
     * Задаёт список областей для выпадающего списка.
     */
    public void setRegions(ObservableList<Region> regions) {
        regionBox.setItems(regions);
    }

    /**
     * Задаёт район, информацию о котором будем менять.
     */
    public void setDistrict(District district) {
        this.district = district;

        nameField.setText(district.getName());
        areaField.setText(district.getArea() == 0 ? "" :
formatArea(district.getArea()));
        adminCenterField.setText(district.getAdminCenter());
        headField.setText(district.getHead());
        regionBox.setValue(district.getRegion());
    }

    /**
     * Returns true, если пользователь кликнул OK, в другом случае false.
     */
    public boolean isOkClicked() {
        return okClicked;
```

```
    }

    /**
     * Вызывается, когда пользователь кликнул по кнопке OK.
     */
    @FXML
    private void handleOk() {
        if (isInputValid()) {
            district.setName(nameField.getText().trim());
            district.setArea(parseArea(areaField.getText()));
            district.setAdminCenter(adminCenterField.getText().trim());
            district.setHead(headField.getText().trim());
            district.setRegion(regionBox.getValue());

            okClicked = true;
            dialogStage.close();
        }
    }

    /**
     * Вызывается, когда пользователь кликнул по кнопке «Отмена».
     */
    @FXML
    private void handleCancel() {
        dialogStage.close();
    }

    /**
     * Проверяет пользовательский ввод в текстовых полях.
     *
     * @return true, если пользовательский ввод корректен
     */
    private boolean isInputValid() {
        String errorMessage = "";

        if (isEmpty(nameField)) {
            errorMessage += "Не указано название района!\n";
        }
        if (regionBox.getValue() == null) {
            errorMessage += "Не выбрана область!\n";
        }
        if (isEmpty(areaField)) {
            errorMessage += "Не указана площадь!\n";
        } else {
            try {
                double area = parseArea(areaField.getText());
                if (area <= 0) {
                    errorMessage += "Площадь должна быть больше нуля!\n";
                } else if (regionBox.getValue() != null && area >=
regionBox.getValue().getArea()) {
                    errorMessage += "Площадь района должна быть меньше площади области
("
                            + formatArea(regionBox.getValue().getArea()) + " км²)!\n";
                }
            } catch (NumberFormatException e) {
                errorMessage += "Площадь должна быть числом!\n";
            }
        }
        if (isEmpty(adminCenterField)) {
            errorMessage += "Не указан административный центр!\n";
        }
        if (isEmpty(headField)) {
            errorMessage += "Не указан глава района!\n";
        }

        if (errorMessage.isEmpty()) {
            return true;
```

```
        } else {
            // Показываем сообщение об ошибке.
            Alert alert = new Alert(AlertType.ERROR);
            alert.initOwner(dialogStage);
            alert.setTitle("Неверно заполнены поля");
            alert.setHeaderText("Пожалуйста, исправьте поля");
            alert.setContentText(errorMessage);
            alert.showAndWait();
            return false;
        }
    }

    private static boolean isEmpty(TextField field) {
        return field.getText() == null || field.getText().trim().isEmpty();
    }

    private static double parseArea(String text) {
        return Double.parseDouble(text.trim().replace(',', '.'));
    }

    private static String formatArea(double area) {
        return area == Math.floor(area) ? String.valueOf((long) area) :
String.valueOf(area);
    }
}
```

Результаты работы программы

Работа приложения показана на рисунках 1–11.

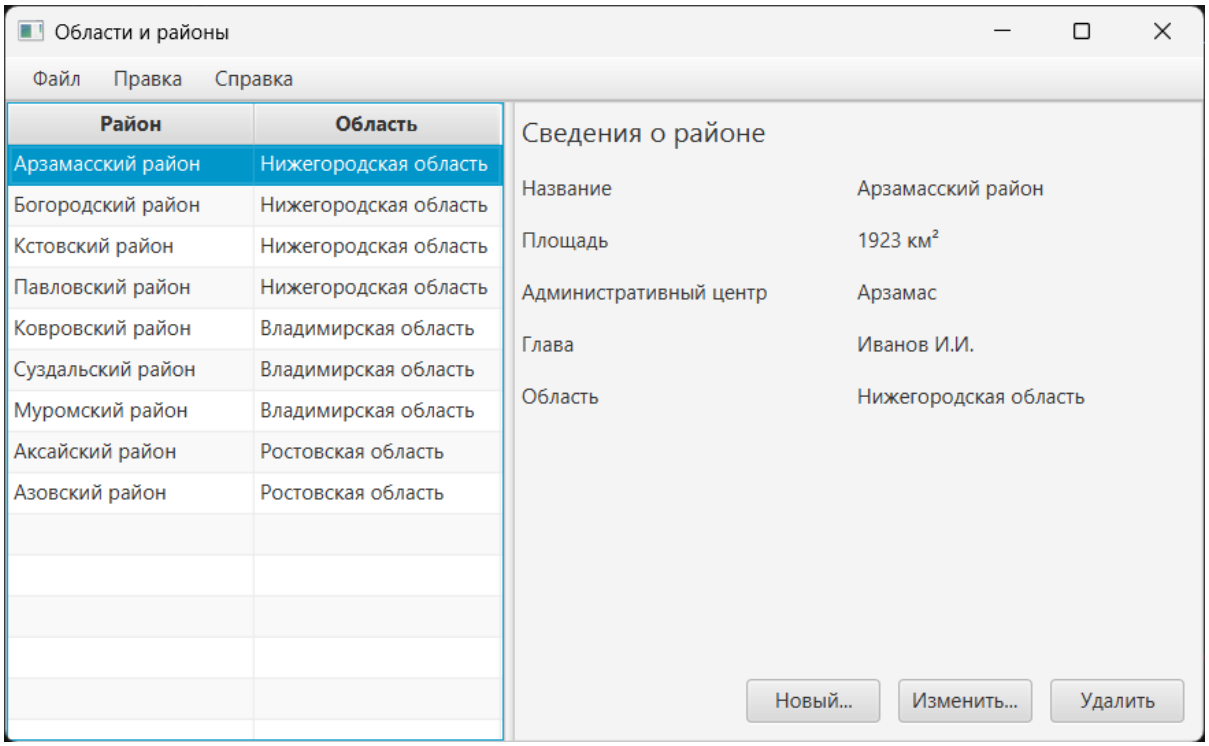


Рисунок 1 – Отображение сведений о выбранном районе

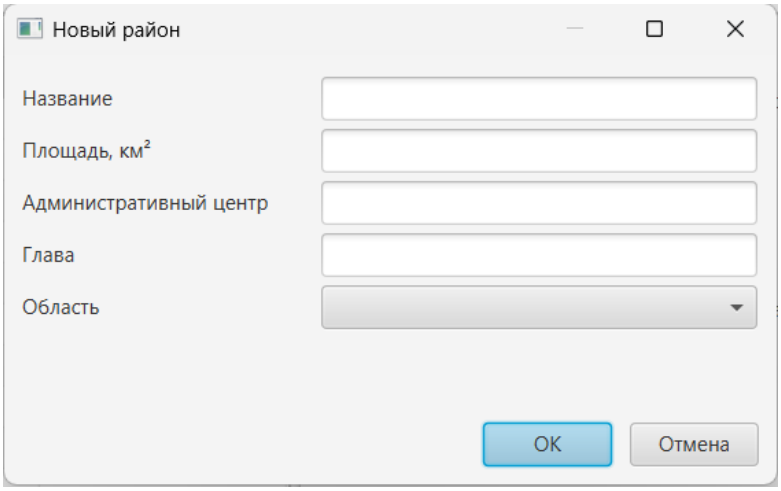


Рисунок 2 – Диалоговое окно добавления нового района

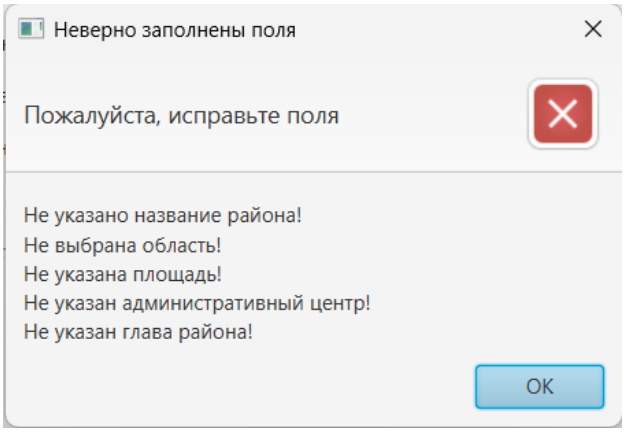


Рисунок 3 – Сообщение об ошибках при незаполненных полях

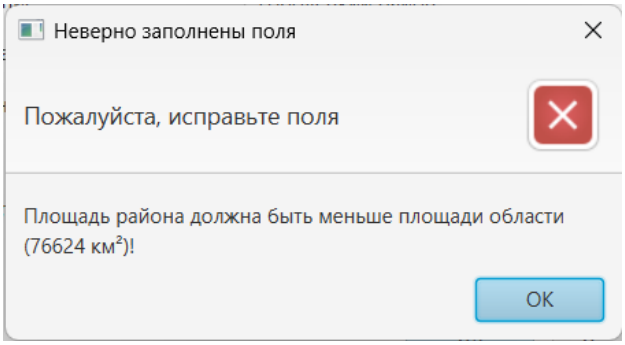


Рисунок 4 – Сообщение об ошибке: площадь района больше площади области

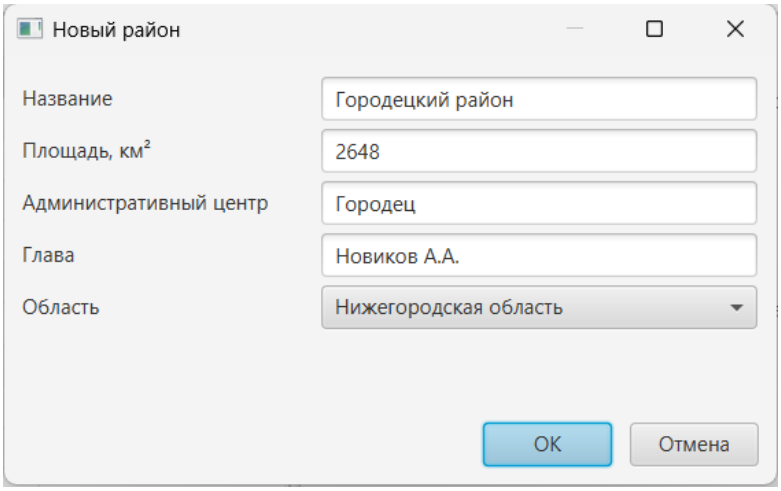


Рисунок 5 – Заполненное окно нового района

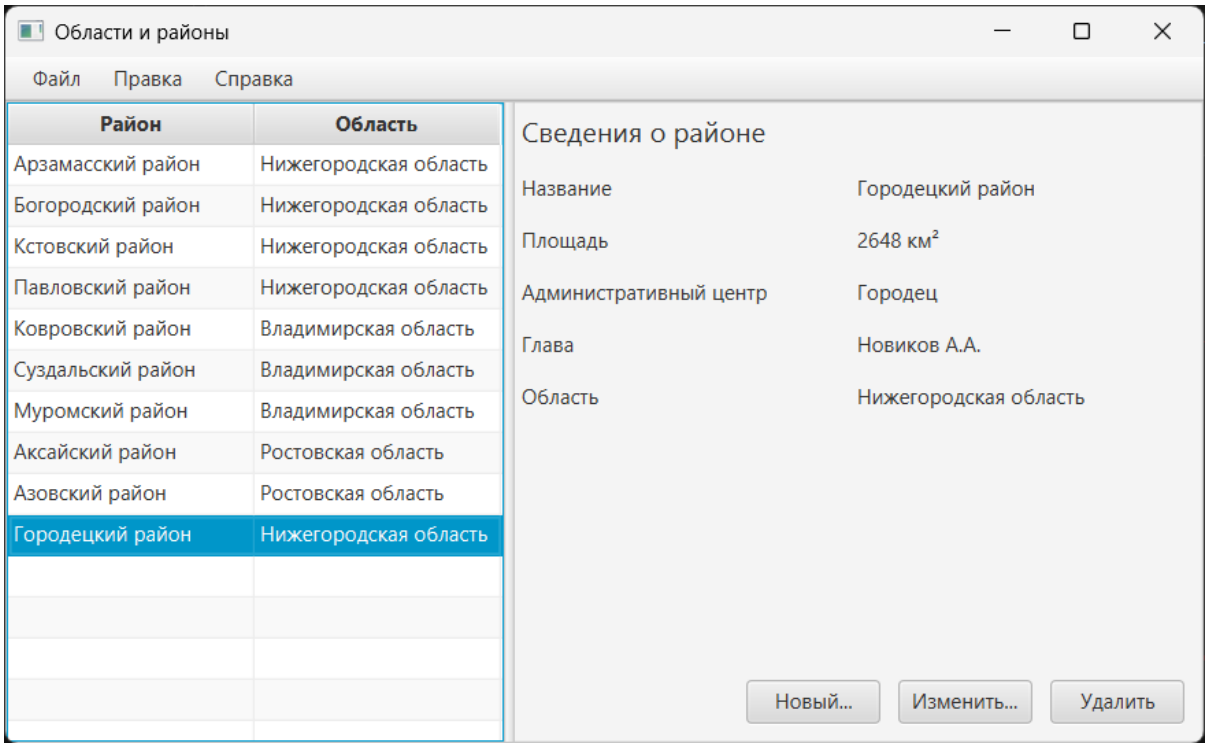


Рисунок 6 – Добавленный район в таблице

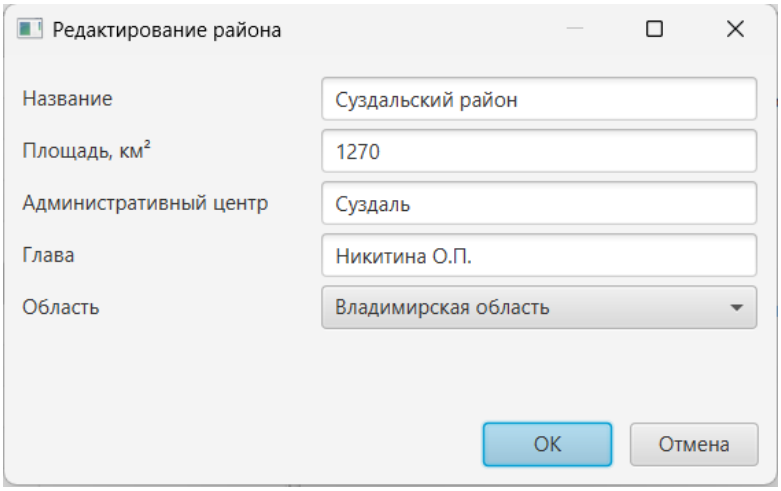


Рисунок 7 – Редактирование сведений о районе

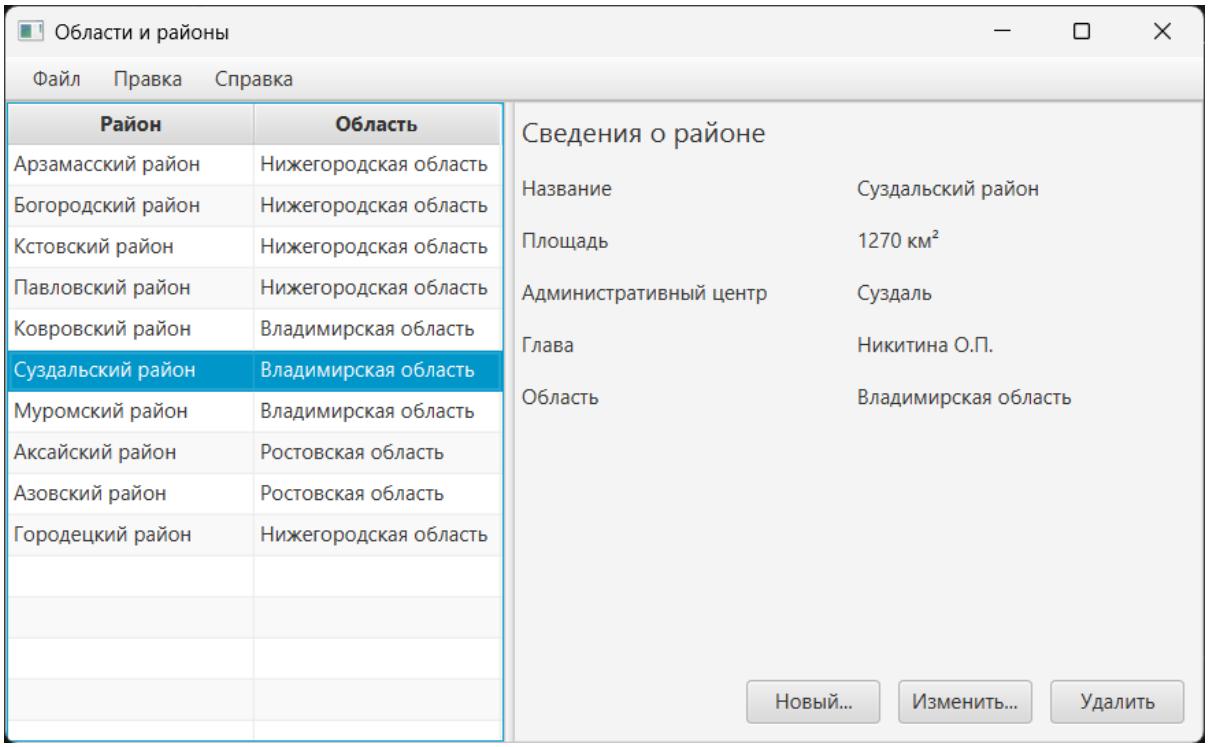


Рисунок 8 – Сведения о районе после изменения

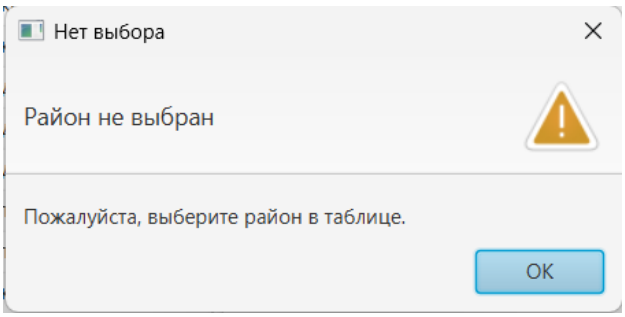


Рисунок 9 – Предупреждение при попытке удаления без выбора района

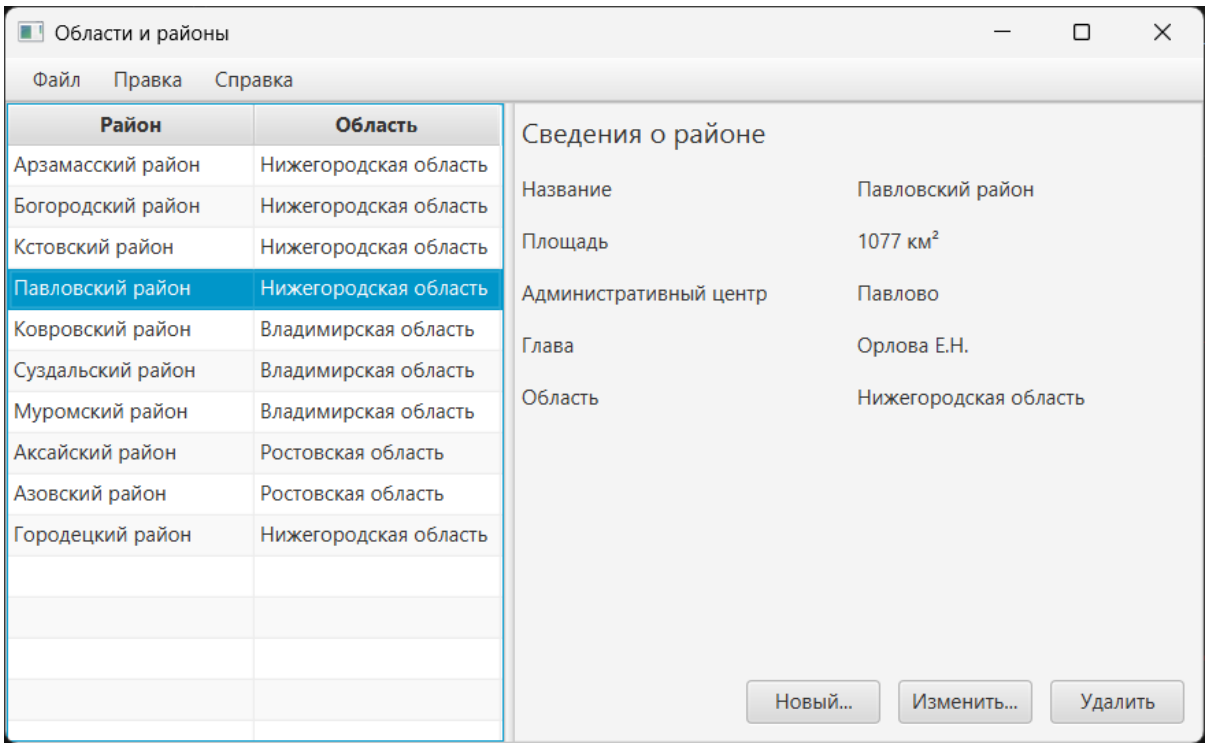


Рисунок 10 – Выбор района для удаления

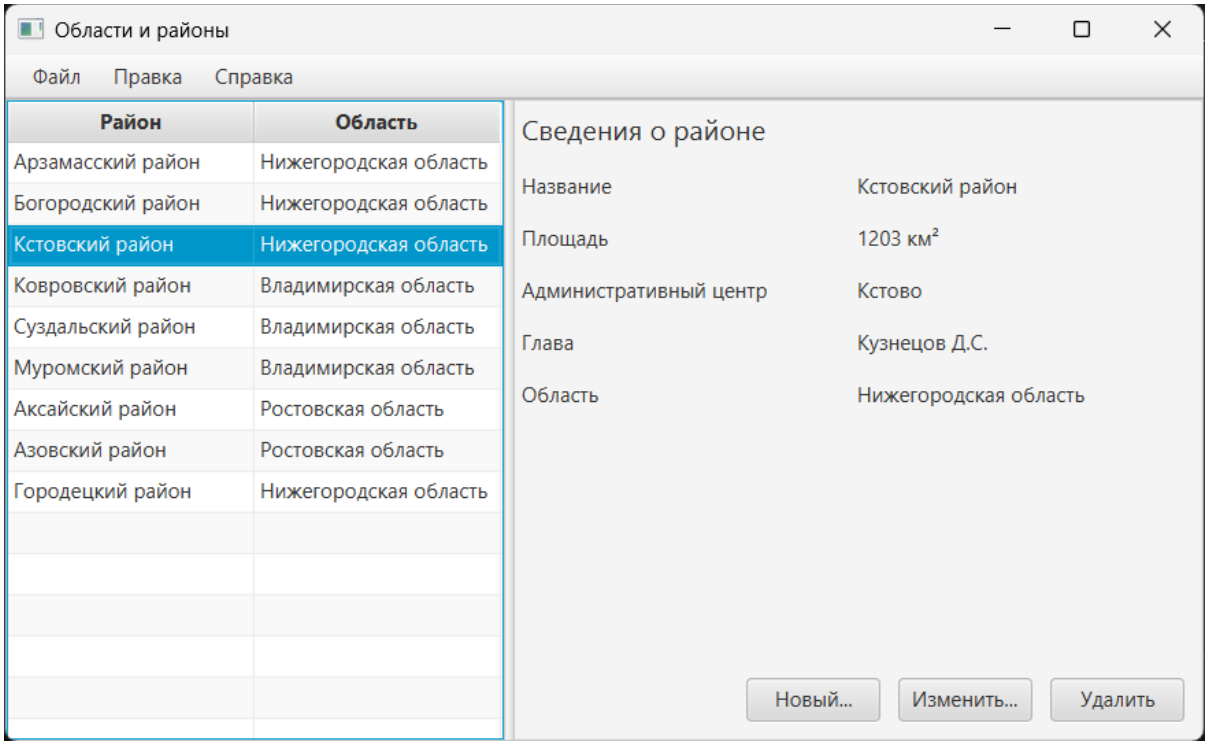


Рисунок 11 – Таблица после удаления района

Вывод

В ходе выполнения лабораторной работы приложение было дополнено обработкой действий пользователя: при выборе района в таблице отображаются его сведения, кнопки «Новый», «Изменить» и «Удалить»

позволяют добавлять, редактировать и удалять районы. Для ввода данных создано модальное диалоговое окно с выпадающим списком областей, а проверка пользовательского ввода не позволяет сохранить незаполненные поля, нечисловую или некорректную площадь.